ChatGPT

Phase II Research Findings: Maritime-Edge-AI

Axis 1 – Sentinel-1 GRD Preprocessing

- **Product format:** A Sentinel-1 Level-1 GRD product is delivered as a `.SAFE` directory containing a `manifest.safe` file, annotation XML, and subfolders for calibration, noise, RFI (if any), and a `measurement` folder with one GeoTIFF per polarization【111†L551-L553】. For each polarization channel (e.g. VV or VH) the GRD product has a corresponding 16-bit GeoTIFF image.
- **Polarizations:** In maritime IW mode, Sentinel-1 typically uses vertical polarization. Products come either **single-pol VV** (code "SV") or **dual-pol VV+VH** (code "DV")【111†L613-L621】. Thus a dual-pol product provides two channels (co- and cross-polarization). (For completeness, Sentinel-1 also supports HH/HV polarizations, but WWJPEG targets used VV/VH.)
- **Data type/dynamic range:** Pixel values in the GRD GeoTIFF are 16-bit integer "detected magnitude" (unsigned)【111†L551-L553】【13†L1848-L1854】. These represent raw SAR amplitude counts. To obtain calibrated backscatter (sigma0), the sensor-specific calibration LUTs (in the calibration XML) must be applied【40†L254-L263】. After calibration, values are often converted to logarithmic (dB) scale for analysis. A typical backscatter range in linear units spans from near 0 up to tens of thousands; in dB this may be roughly –40 to 0 dB depending on scene brightness.
- **Spatial resolution:** IW GRD products have 10 m pixel spacing in both range and azimuth【13†L1890-L1894】 (≈20 m ground resolution). This matches the resolution of iVision-MRSSD/SSDD training data.
- **Calibration:** *Radiometric calibration* (to sigma0) is recommended. Sentinel's calibration XML provides conversion factors so that $$\sigma^0 = 10\log_{10}(DN^2 + e)$$ (DN = digital number). While YOLO detection could operate on raw DNs, consistent calibration ensures intensities match the distributions seen in training. In practice, apply the calibration factors (and thermal noise removal) early in preprocessing【40†L254-L263】.
- **Speckle filtering:** SAR images have granular noise. Applying a mild speckle filter (e.g. 5×5 Lee) can smooth background clutter, but it also blurs small features. Many operational ship-detection pipelines **skip heavy filtering** to avoid degrading small targets【44†L13-L21】. The Copernicus GRD workflow includes an optional filter step, but advises disabling it if fine detail (like small vessels) is critical【44†L13-L21】. Thus speckle filtering is *optional*: if used, choose a conservative filter (Lee or Frost).
- **Normalization:** The YOLO models were trained on 8-bit images. The iVision-MRSSD dataset was created by exporting SAR images to high-quality JPEG, using histogram equalization and brightness/contrast adjustments to enhance vessel visibility【46†L790-L798】. To match this, calibrated GRD data should be normalized to [0,255]. A practical approach is to convert sigma0 to dB, then linearly map a chosen dB range (e.g. –30 dB to 0 dB) into [0,255], with optional clipping. Alternatively, apply histogram equalization or contrast stretching (as done in iVision-MRSSD【46†L790-L798】) so that ships stand out. The key is to reproduce the roughly 8-bit contrast distribution of the training set.
- **Tiling:** The trained models expect fixed-size patches. In iVision-MRSSD, images were sliced into **512×512 tiles with 50% overlap**【46†L798-L806】. For inference, a similar tiling (e.g. 512×512 or 640×640 with ~50% overlap) is recommended to cover large scenes without missing objects at patch boundaries【46†L798-L806】. (Overlap allows YOLO to detect ships that straddle two patches.)

- **Training-data compatibility:** We must ensure Sentinel-1 inputs are processed similarly to the training data. iVision-MRSSD and SSDD images were enhanced with histogram equalization, contrast/brightness adjustments, and noise reduction【46†L790-L798】. We will replicate those steps so that the model sees a familiar intensity distribution. If the original training used only amplitude (not calibrated sigma0), we might skip full calibration or apply the same log scaling. In summary, apply the same normalization/enhancement pipeline as in [46] to make real data "look like" the training JPEGs.

Feasibility: *Moderate to complex.* The required steps (reading SAFE, calibrating, filtering, normalizing, and tiling) can be done with Python (e.g. `rasterio`, `numpy`) without SNAP. Calibration uses small lookup tables (lightweight). However, ensuring the processed Sentinel-1 intensity distribution matches the JPEG training data exactly is tricky and may require experimentation. Implementing speckle reduction on a CPU is also moderately expensive. This axis is feasible but requires careful tuning of the preprocessing chain.

Uncertainties: The largest unknowns are: (1) **Effect of calibration vs. raw values.** If we calibrate to sigma0, the range may differ from the uncalibrated data in MRSSD/SSDD. We may need to empirically adjust scaling. (2) **Speckle filtering impact.** We don't know how much filtering helps/hurts YOLO performance. (3) **Normalization bounds.** Choosing the optimal dB-to-8bit mapping or equalization could affect contrast. (4) **Overlap trade-offs.** The chosen tile size and overlap should balance GPU memory vs. detection completeness. These will require experimental validation.

Axis 2 – Copernicus Data Space API

- **OData search endpoint:** Sentinel-1 products can be searched via the Copernicus Data Space *catalog* OData API at: `https://catalogue.dataspace.copernicus.eu/odata/v1/Products`. Queries support filters on collection name, product type, date, and spatial area. For example, to find IW GRD products over Morocco in June 2025, one could use a query like:

```
https://catalogue.dataspace.copernicus.eu/odata/v1/Products?
  $filter=Collection/Name eq 'SENTINEL-1'
    and Attributes/OData.CSC.StringAttribute/any(att: att/Name eq
'productType' and att/OData.CSC.StringAttribute/Value eq 'IW_GRDH_1S')
    and ContentDate/Start ge 2025-06-01T00:00:00Z
    and ContentDate/Start le 2025-06-30T23:59:59Z
    and OData.CSC.Intersects(area=geography'SRID=4326;POLYGON((-17 27,-1
27,-1 36,-17 36,-17 27))')
```

Here `OData.CSC.Intersects` is used to filter by bounding box【62†L850-L856】. The API returns JSON results including fields like `Id`, `Name`, `S3Path`, and geographic footprint【62†L858-L866】. (In the JSON, `Id` is a UUID, and `S3Path` is a data URL.) The example above builds on Sentinel Hub's documented pattern【62†L850-L859】.

- **JSON response fields:** A typical response entry has fields:
- `Id`: product UUID (for download)
- `Name`: product identifier (e.g. `S1A_IW_GRDH_1SDV_...`)
- `ContentType`, `ContentLength` (if present)

- `S3Path`: the path on the Copernicus storage (usable for downloads)【62†L858-L866】
- `Footprint` / `GeoFootprint`: WGS84 polygon of coverage.
- `ContentDate`: start/end timestamps.
  These cover the product metadata.
- **Example query:** Using the above template, an example query for Sentinel-1 over the Moroccan box might be:

```
https://catalogue.dataspace.copernicus.eu/odata/v1/Products?
  $filter=Collection/Name eq 'SENTINEL-1'
    and OData.CSC.Intersects(area=geography'SRID=4326;POLYGON((-17 27,-1
27,-1 36,-17 36,-17 27))')
    and ContentDate/Start ge 2025-06-01T00:00:00Z
    and ContentDate/Start le 2025-06-30T23:59:59Z
```

(Add `productType` filters as needed.) The *Copernicus Data Space* documentation provides similar examples【62†L850-L859】【62†L858-L866】.

- **Authentication (Keycloak):** Programmatic access requires an OAuth token. The CDSE uses Keycloak; the standard method remains `grant_type=password` with `client_id=cdse-public` (as documented)【67†L799-L807】. This is still valid in 2024/25. A user must POST credentials to `https://identity.dataspace.copernicus.eu/auth/realms/CDSE/protocol/openid-connect/token` to get a bearer token. (No recent changes have occurred beyond this scheme.)
- **Download endpoint:** Once an `Id` is known, the product can be downloaded via the *Zipper* API:

```
GET https://zipper.dataspace.copernicus.eu/odata/v1/Products(<Id>)/$value
```

Include the bearer token in the `Authorization` header. This returns the zipped `.SAFE` (binary content)【69†L1-L4】. For example:

```
https://zipper.dataspace.copernicus.eu/odata/v1/Products(acdd7b9a-
a5d4-5d10-9ac8-554623b8a0c9)/$value
```

(An example from documentation【69†L1-L4】 shows this pattern with a sample UUID.)

- **Product size:** A full Sentinel-1 IW GRD product (dual-pol) is typically on the order of **~100–300 MB** (zipped). (Exact size varies by orbit length and polarization.) This volume is moderate but non-trivial for an edge microservice. Downloading entire scenes repeatedly may be slow. In practice, consider retrieving only needed polarizations or time segments.

- **Spatial subsetting:** The CDSE OData does *not* support subscene cropping natively. To avoid full-scene downloads, one option is to use external processing services. For instance, **Sentinel Hub's Process API** can generate on-the-fly SAR subsets for a specified AOI (it automatically stitches tiles to cover any area)【107†L26-L34】. You could write a small evalscript to fetch just the coastal region of interest (e.g. 12 NM zone). Alternatively, openEO-based services (EODC, etc.) allow requesting Sentinel-1 with a spatial bounding box. Without such services, the microservice would have to download the full product and then perform a cut. (There is no known COG option for Copernicus GRD, so HTTP range requests on the SAFE are not practical.)

- **Quotas and limits:** Free CDSE accounts have usage limits. As of mid-2025, the **monthly download cap is 12 TB**【76†L813-L821】; after that, download speed is throttled. OData catalog queries can be heavy, but real-time limits are generous (2000 requests/min)【76†L805-L813】. If using Sentinel Hub, note its free tier allows ~10,000 API calls per month and ~10,000 Processing Units【76†L805-L813】. (Sentinel Hub Process API PUs are consumed per request depending on area and resolution.) In summary: CDSE is virtually unlimited for metadata queries, but watch the 12 TB/month bandwidth quota.

Feasibility: *Moderate.* All required APIs exist and are well-documented【62†L850-L859】【67†L799-L807】. Writing code to query OData and fetch downloads is straightforward with standard HTTP libraries. The main challenges are token management and large file handling. Downloading full GRD files is viable but heavy; combining OData filters and external APIs can minimize data volumes.

Uncertainties: No built-in CDSE support for spatial subsetting – we depend on third-party APIs (Sentinel Hub/openEO) for efficient ROI crops. Also, API quotas (especially if many users or large areas) could become restrictive if usage scales. Finally, the latest status of Keycloak endpoints should be monitored (but it is stable as of now).

Axis 3 – SatNOGS TLE & Telemetry Integration

- **TLE retrieval:** SatNOGS DB provides an open REST API (`https://db.satnogs.org/api/`) to query satellite data. For example, querying by NORAD ID returns the current Two-Line Element set for that satellite. For SNUGLITE (NORAD 43784), the web interface shows:

```
1 43784U 18099AC  26175.08876981  .00010485  00000-0  39458-3 0  9991
2 43784  97.3868 229.3010 0001458 236.9546 123.1553 15.27306223414379
```

(This is the "Latest TLE" on the SNUGLITE page【96†L241-L242】.) Via the API one would see these lines in JSON fields (e.g. `tle1`, `tle2`). The TLE source is Space-Track (as shown) and is updated regularly (e.g. daily). **Note:** SatNOGS does *not* offer historical TLE lookups via the API; only the latest orbit elements are available【82†L75-L78】. For archival TLEs one must use an external service like Space-Track. The API calls themselves do not require authentication for reading satellite TLE.

- **TLE format:** The API returns the two-line elements as plain text strings. Users should ensure they query by satellite ID or NORAD number, for example:

```
GET https://db.satnogs.org/api/satellites?norad_cat_id=43784
```

(This would return JSON including the latest `tle1` and `tle2` lines for that satellite.) The lines match what is displayed on the SatNOGS DB page【96†L241-L242】. The response also includes the timestamp of the TLE update.

- **Telemetry/beacon data:** SatNOGS records raw telemetry frames for each observed transmitter. These frames (packets) are stored in raw hex form in the database. To interpret them, one needs decoders. The SatNOGS team provides Kaitai-based decoding schemas and Python interfaces【82†L87-L91】. For example, SNUGLITE's telemetry includes GPS position, velocity, magnetometer, and power subsystem readings (as seen in its Kaitai decoder fields). A user can either retrieve raw data and decode it offline, or use the SatNOGS Dashboard/DataWarehouse which automatically decodes and stores values in a time-series database. As noted on the SatNOGS forum, the raw data are freely accessible, but decoding requires knowledge of the satellite's protocol【82†L87-L91】.

- **API for telemetry frames:** The SatNOGS *Network* API provides direct access to downloaded observation data. For example, one can fetch all demodulated data for a given satellite via:

```
https://network.satnogs.org/api/data/?satellite__norad_cat_id=40379
```

This returns JSON packets of raw payloads and timestamps for that satellite【103†L69-L77】. Filters (e.g. date range, ground station) can be applied as query parameters. (Users have noted that this *network* API is the practical way to bulk-download frames【103†L69-L77】.) The SatNOGS DB itself also offers CSV exports per satellite but not a simple bulk-API; the network endpoint is preferable for programmatic access. Once frames are obtained, the associated ground-truth telemetry fields can be extracted using the decoders (or by reading the SatNOGS Dashboard, which already decodes and stores them【103†L53-L61】).

- **Content of telemetry:** The telemetry contents vary by CubeSat. In general, decoded telemetry may include temperatures, voltages, currents, GNSS data, orientation, etc. For SNUGLITE, the decoded fields include position (latitude, longitude, altitude), velocity, battery voltages/currents, magnetometer readings, and power switch states. These decoded values are available on the SatNOGS Dashboard (influxDB) for each pass【103†L53-L61】. If no decoder is available for a satellite, the raw frames can still be retrieved but will remain as hex strings.

Feasibility: *Moderate.* Retrieving the latest TLE is straightforward via the open SatNOGS API (no auth) and yields immediate results. Retrieving telemetry frames is also feasible using the network API (as JSON)【103†L69-L77】. The main work is decoding the frames. Fortunately, the SatNOGS decoders (Kaitai schemas) are open-source【82†L87-L91】 and can be integrated into a Python pipeline. Accessing timeseries (via the Dashboard) may require building an account or setting up InfluxDB (the SatNOGS Dashboard uses InfluxDB to store telemetry【103†L53-L61】).

Uncertainties: Not all satellites have decoders uploaded, so telemetry availability can be spotty. The volume of data (especially if collecting many passes) could be large; efficient filtering by date is needed (the network API allows date filters). Also, without a dedicated export API, one may need to paginate through

results. Finally, syncing timeseries with the imagery (to know when the satellite was overhead) may require careful matching of timestamps from the two systems.

Axis 4 – Microservice Feasibility

- **Preprocessing time (CPU):** Fully processing a Sentinel-1 IW scene (~250×250 km) on CPU is time-consuming. For reference, users report that running radiometric calibration, terrain-correction, etc. in SNAP took minutes per scene (~800 scenes took ~2 days)【105†L13-L19】. A lean Python routine (using `rasterio`/`numpy`) will be faster than SNAP's full workflow, but we should expect *seconds-to-minutes per 10k×10k image tile*. Speckle filtering (convolution) and calibration (per-pixel math) are the heaviest steps. In a Docker microservice with ~2 GB RAM and no GPU, one full GRD (all bursts) might take on the order of **a few minutes** of CPU time (depending on size of ROI). This is acceptable if done on-demand or scheduled, but not "instant."
- **Spatial crop strategy:** To reduce workload, the microservice should **download and process only the area of interest** whenever possible. As noted, CDSE itself cannot subset, but one can call a service like Sentinel Hub (or openEO) to retrieve just the coastal band. Sentinel Hub's Processing API, for example, will return SAR data clipped to an arbitrary AOI (and handle mosaicking)【107†L26-L34】. If such services are unavailable, the microservice could download the full scene and then extract the ROI with `rasterio` windows (more costly). For geocoded tasks, using Cloud-Optimized GeoTIFFs (COGs) would be ideal, but native Copernicus GRD are not COGs. One could convert downloaded GRD patches to small COGs for internal use, but this adds overhead.
- **Pipeline references:** We are not aware of an existing turnkey repo that exactly does "CDSE→GRD→YOLO detection." However, the AllenAI vessel-detection codebase demonstrates how to download Sentinel-1 data and feed it into ML detectors (it includes a `download_imagery.py` script)【110†L19-L22】. In practice, we will write a custom microservice that: queries CDSE (Axis 2), downloads/receives the appropriate imagery, applies the Axis 1 preprocessing, then calls the YOLO detection model. The continuous-monitoring mode (via API trigger) will reuse this same chain for each new arrival.
- **Inter-service format:** For minimal latency, intermediate data should be compact. Likely, after preprocessing we can send **8-bit PNG** or **JPEG** images to the detector service (YOLOv8) since it was trained on JPEGs. Alternately, we could pass raw NumPy arrays (via shared memory or serialized as `.npy`), but that has more overhead. If geolocation is needed downstream, we should carry along the bounding-box coordinates (separately or in GeoJSON). In a microservice context, sending PNG tiles (with some lossless compression) strikes a good balance of size vs. speed. GeoTIFFs preserve full geoinfo but are large. Numpy arrays (`.npy`) are fast to load but may exceed our RAM budget if many patches queue up.

Feasibility: *Challenging but tractable.* The core algorithms (calibration, filtering, tiling, YOLO inference) are all CPU-friendly and lightweight libraries exist (`rasterio`, `numpy`, `onnxruntime`). The bottleneck is I/O and convolution. However, since the microservice will handle one scene or tile at a time, and we use caching where possible, the load is manageable. Using ROI downloads (Sentinel Hub) can greatly reduce data volumes. GPU inference for YOLO is not available, but the tiny YOLOv8n model in INT8 can run on CPU in real-time for single images.

Uncertainties: The biggest unknown is total **latency** from end-to-end. If a full GRD must be loaded and processed, a few minutes per scene is expected. We will test memory usage and may need to split scenes into bursts. Another question is cloud vs. on-premise: if deployed on KSF.space, we must see what

bandwidth and compute are available. Finally, the robustness of the Dockerized pipeline (error handling, file sizes) will need validation.

Sources: Official Copernicus product specs【111†L551-L553】【13†L1848-L1854】【111†L613-L621】, Sentinel Hub documentation【107†L26-L34】, and SatNOGS API/forum posts【96†L241-L242】【82†L75-L78】【103†L69-L77】.
